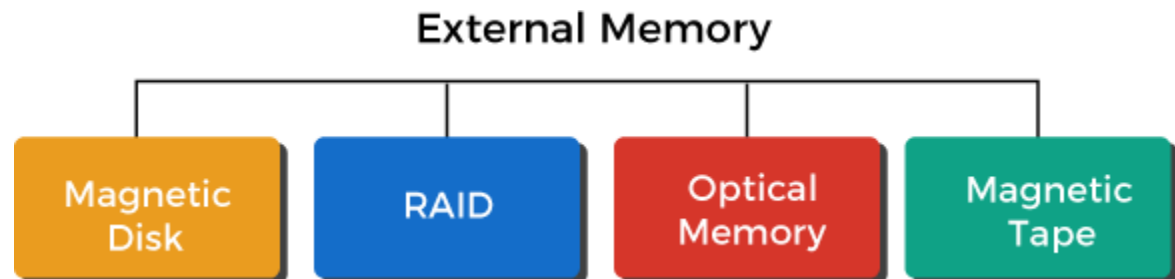


Module 6

Subsystems

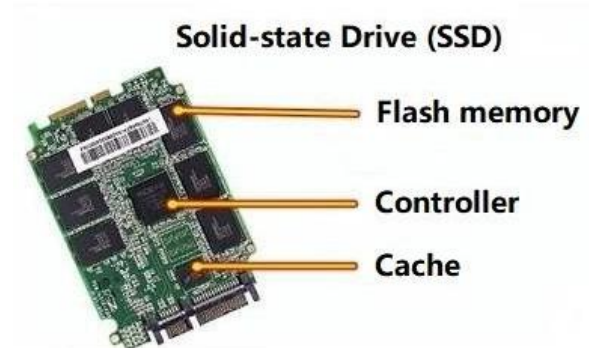
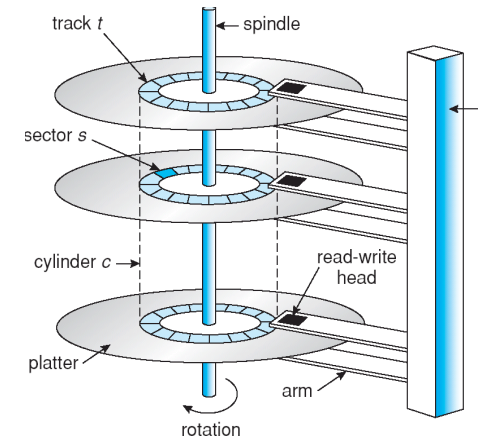
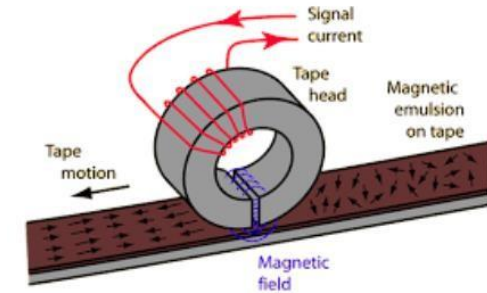
External Storage Systems

- **External memory** - secondary memory or backing store
- Used to store a huge amount of data because it has a **huge capacity**.
- At present, it can measure the data in hundreds of megabytes (MB) / gigabytes (GB) / TB...
- The important property of external memory is that whenever the computer switches off, then **stored information will not be lost**.
- The external memory can be categorized into four types:
 1. Magnetic disk
 2. Magnetic Tape
 3. Optical memory
 4. RAID



External Storage Systems -Examples

- **Magnetic tape**
 - An external storage medium that consists of a loop of flexible celluloid-like material that can store data in the form of tiny magnetic fields.
- **Magnetic disk**
 - A storage device that uses magnetization to write, rewrite, and access data. The disk is divided into platters, which are made up of concentric circles called tracks.
- **Hard disk drive (HDD)**
 - A larger-capacity external storage unit that plugs into a computer or console for long-term storage. Hard drives are ideal for storing files and applications because of their size, capacity, and stability
- **Solid state drive (SSD)**
 - A non-volatile data storage device that can be used internally or externally. SSDs store data on microchips or integrated solid-state flash memory and don't have moving parts.



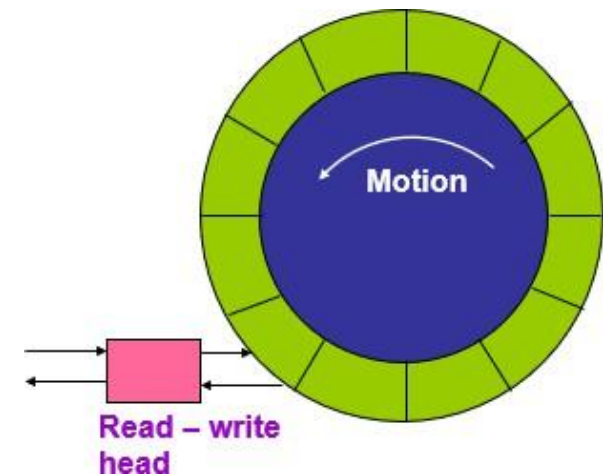
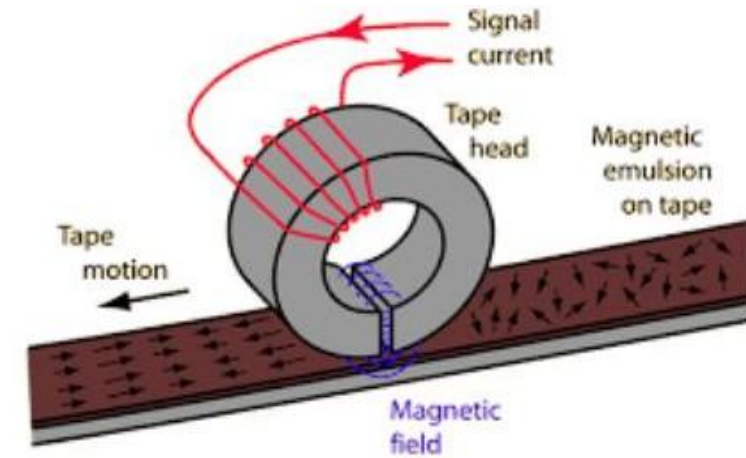
External Storage Systems -Magnetic Technologies

Magnetic Technologies:

Magnetic drums, magnetic tape and magnetic disks are types of magnetic memory. These memories use property of magnetic memory.

1.Magnetic Tape

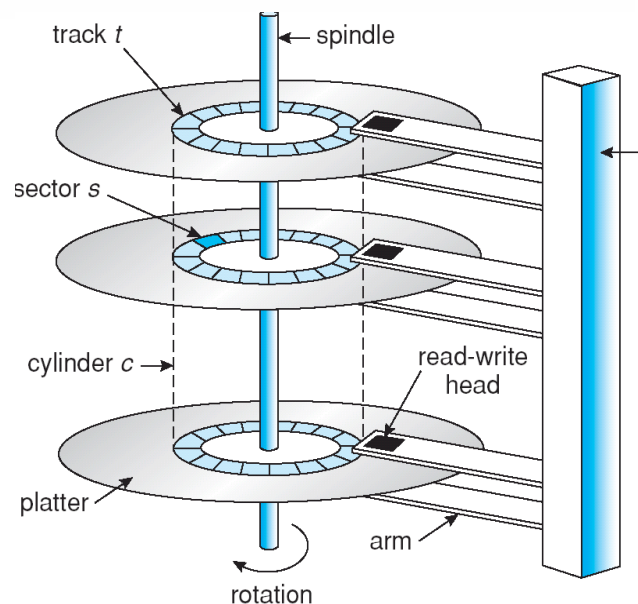
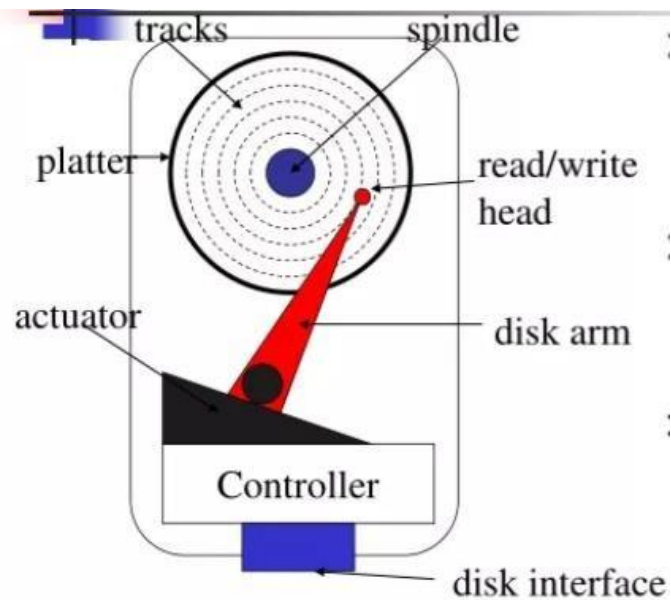
- In **magnetic tape only one side of the ribbon** is used for storing data.
- It is **sequential memory** which contains thin plastic ribbon to store data and coated by magnetic oxide.
- Data read/write speed is **slower** because of sequential access.
- It is **highly reliable** - requires magnetic tape drive writing and reading data.



2. Magnetic Disk

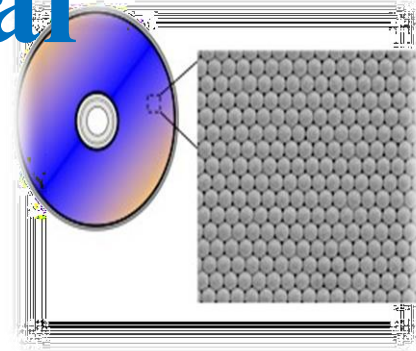
- A magnetic disk is a storage device that **uses a magnetization process** to read, write, rewrite and access data.
- A type of secondary memory that is a **flat disc covered with a magnetic coating to hold information**.
- The Magnetic disk is made of a **set of circular platters**.
 - It is covered with a magnetic coating and stores data in the form of **tracks, spots, and sectors**.
- **Less expensive** than RAM
- Can store **large amounts of data**
- **Data access rate is slower** than main memory because of secondary memory.
- Data can be **modified or can be deleted easily** in the magnetic disk memory.
- It also **allows random access** to data.





- A **Hard disk Drive (HDD)** includes two main elements =>
 - a spinning platter
 - an actuator arm.
- The **platter** is a circular magnetic disk containing tracks and sectors that retain data.
 - **“Platter”** – Circular Piece of Magnetic Material – Coated Aluminium
 - One Platter Has **2 Sides** - Each Side is Divided into Concentric Rings Called **“Tracks”** - Each Track is Divided into Arcs Called **“Sectors”**.
 - The standard size of each sector is 512Bytes (Note: 512 bytes is a universal size)
- The **actuator arm** moves across the platter to read and write data.

External Storage Systems-Optical Technologies

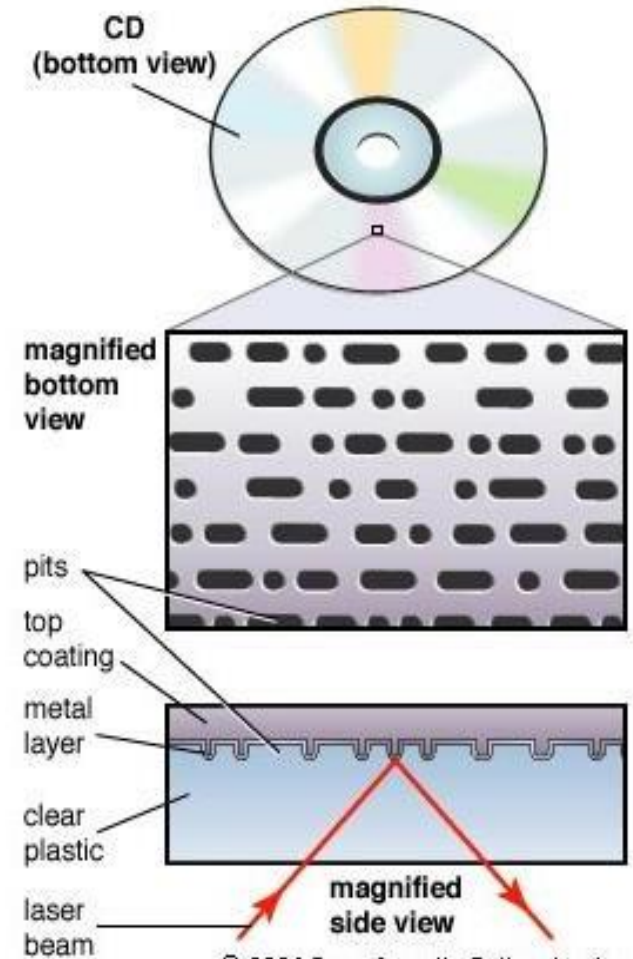


3. Optical Disc

- The Optical memory is an **electronic storage medium** that **uses a laser beam to store and retrieve digital (binary) data**.
- In optical storage technology, a **laser beam encodes digital data on an optical disc** or laser disc in the form of tiny pits arranged in a spiral pattern on the surface of the disc.
- also referred to as an optical disk, optical storage, optical media, Optical disc drive, disc drive, **which reads and writes data by using optical storage techniques and technology**.
- **Examples:** Digital Versatile Disk (DVD), Compact Disk (CD) and Blu-Ray Disk

Working of optical drive

- The spindle spins the optical media (CD, DVD, Blu-ray)
- A laser shines through a lens and reflects off the bottom of an optical disc
- If the laser shines in a pit, the computer reads a “zero”
- If the laser reflects off a land, the computer reads a “one”



External Storage Systems-Solid State Drivers (SSD)

- Solid-state drive (SSD) is a solid-state storage device that uses integrated circuit assemblies as memory to store data.
- SSD is also known as a solid-state disk although SSDs do not have physical disks.
- There are no moving mechanical components in SSD.
- This makes them different from conventional electromechanical drives such as Hard Disk Drives (HDDs) or floppy disks, which contain movable read/write heads and spinning disks.
- It is a type of non-volatile memory that retains data even when power is lost.

Classification of SSD:

1. Flash-based solid-state drives

- Flash-based solid-state drives use FLASH chips as storage media, which is also commonly referred to as SSD. Its appearance can be made into a variety of shapes, such as notebook hard drives, micro hard drives, memory cards, U disks, and other styles.

2. DRAM-based solid-state drives

- DRAM-based solid-state drives used DRAM as a storage medium, and the application range is narrow. It imitates the design of a traditional hard disk, which can be used for volume setting and management by most operating system file system tools, and provides industry-standard PCI and FC interfaces for connecting to a host or server.

Architecture of SSD:

Three components: Controller, Cache, Flash Memory

1.Controller

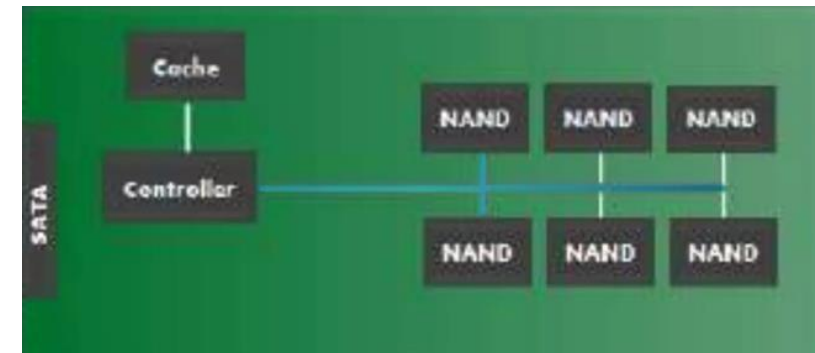
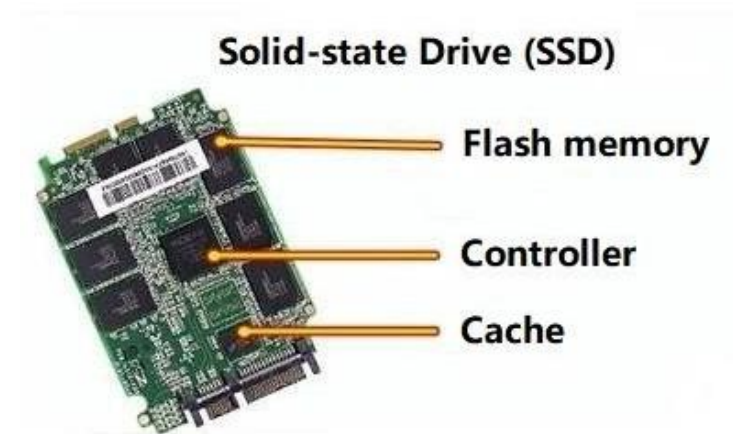
The controller is the **brain of the solid-state drive**. One of its functions is to **rationally allocate the data load** on each flash memory, and the other is to undertake the entire data transfer, connecting the flash memory chip and the external SATA interface.

2.Cache

Next to the controller chip are cache particles. Like traditional hard drives, solid-state drives require a **high-speed cache chip to assist** the main controller for data processing.

3.Flash memory

Except for the controller and the cache, most of the other positions on the PCB board are **NAND flash memory chips**.



Advantages of SSD:

- Faster read/write speeds. SSDs can access large files quickly.
- Quicker boot times and better performance. Because the drive does not need to spin up as an HDD would, it is more responsive and provides better load performance.
- Durability. SSDs are more shock-resistant and can handle heat better than HDDs because they do not have moving parts.
- Power consumption. SSDs need less power to operate than HDDs due to their lack of moving parts.
- Quieter. SSDs produce less audible noise because there are no moving or spinning parts.
- Size. SSDs come in a variety of form factors whereas HDD sizes are limited.

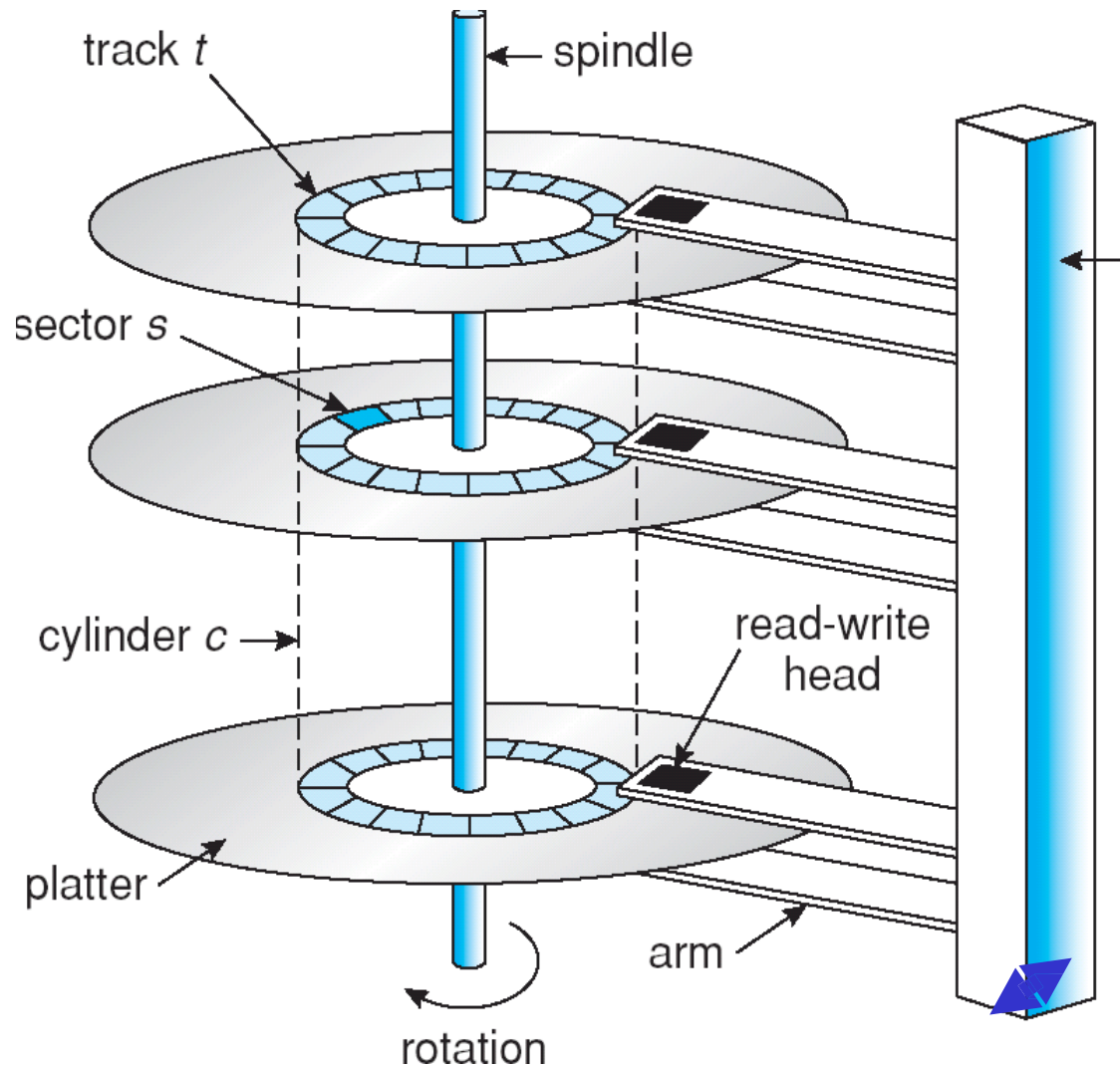
Disadvantages of SSD:

- Cost. SSDs are more expensive than traditional HDDs.
- Life expectancy. Some SSDs, for example, those using NAND memory-flash chips, can only be written a specified number of times that is typically less than HDDs.
- Performance. Limitations on the number of write cycles cause SSDs to decrease in performance over time.
- Storage options. Because of cost, SSDs are typically sold in smaller sizes.
- Data recovery. This time-consuming process can be expensive, as the data on damaged chips may not be recoverable.

SSD Vs HDD

Solid-State Drive	Hard Disk Drive
Random access time 0.1 ms.	Random access time 5-10 ms.
Read latency time very low.	Read latency time high.
100MB/s to 500MB/s.	50MB/s to 100MB/s.
High Reliability.	Low reliability.
SSDs have no moving parts to fail mechanically.	HDDs have moving parts and are subject to sudden failure.
Small and light weight.	Relatively large and heavy.
Till now SSDs are available in size up to 2TB.	Till now HDDs are available up to 4TB .
Power Consumption 2watts	6-12 watts.

Organization and Structure of Disk Drives



Typical parameters :

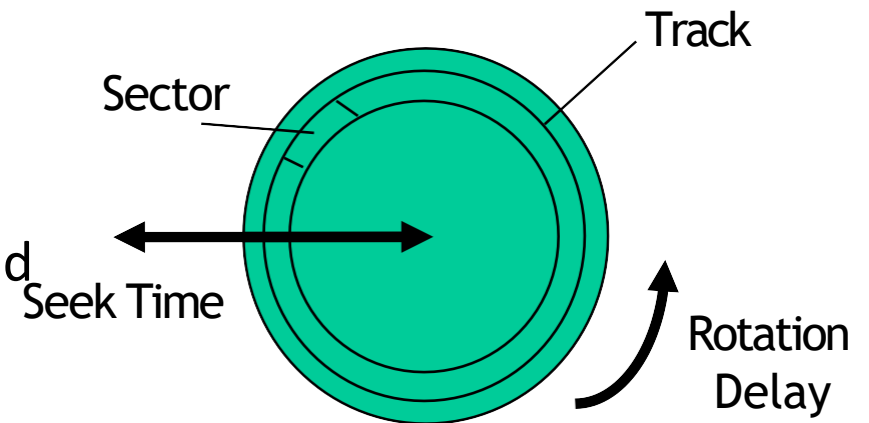
- 1 spindle
- 1 arm assembly
- 1-2 sides/platter
- 1 head per side
(but only 1 active head at a time)
- 700-20480 tracks/surface
- 16-1600 sectors/track

- **Accessing a disk requires:**

- specify sector: C (cylinder), H (head), and S (sector)
- specify size: number of sectors to read or write
- specify memory address

- **Performance:**

- seek time: move the arm assembly to track
- Rotational delay: wait for sector to come around
- transfer time: get the bits off the disk
- Controller time: time for setup



Typical Disk Parameters

Parameter	IBM 360-KB floppy disk	WD 18300 hard disk
Number of cylinders	40	10601
Tracks per cylinder	2	12
Sectors per track	9	281 (avg)
Sectors per disk	720	35742000
Bytes per sector	512	512
Disk capacity	360 KB	18.3 GB
Seek time (adjacent cylinders)	6 msec	0.8 msec
Seek time (average case)	77 msec	6.9 msec
Rotation time	200 msec	8.33 msec
Motor stop/start time	250 msec	20 sec
Time to transfer 1 sector	22 msec	17 μ sec

Compare Seek time, rotation time, transfer time

- **Disk Access**

Design considerations

- Seek time (moving head to correct cylinder) and rotational latency time (waiting for correct sector) are **greater than** data transfer time (time to read)
- Lots of errors possible (bad sectors, bad bits)
- **Caching blocks** that happen to pass under disk head **commonly used**

Example

- Average time to read/write 512-byte sector
- Disk rotation at 10,000 RPM
- Seek time: 6ms
- Transfer rate: 50 MB/sec
- Controller overhead: 0.2 ms
- ***Average time = Seek time + rotational delay + transfer time + controller***

- **Disk Format**

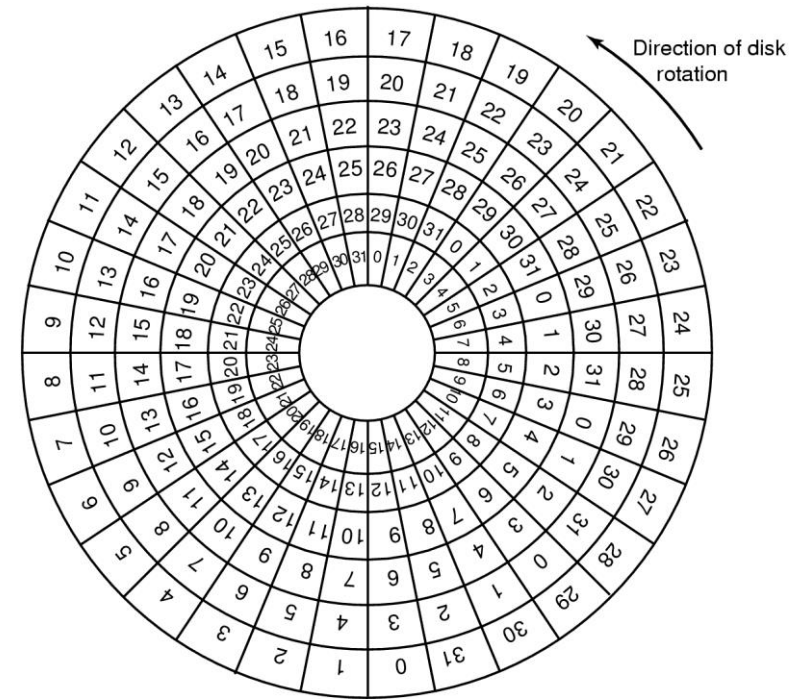
- Typical sector (512 bytes)

- **Preamble** identifying start code and sector address
- **Data**
- **Error correction code (16 bits)** - At least detecting errors possible with probability almost 1



- **Cylinder skew**

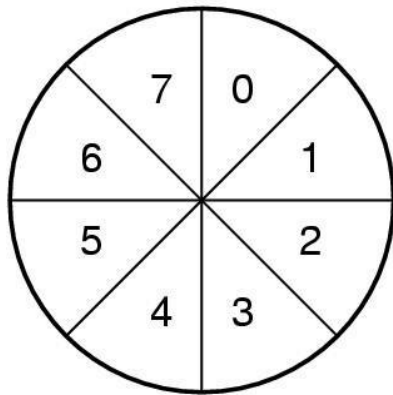
- **After reading an entire track**, as head is moved across a cylinder, seeking next sector
shouldn't cause waiting for full rotation



- **Disk Format**

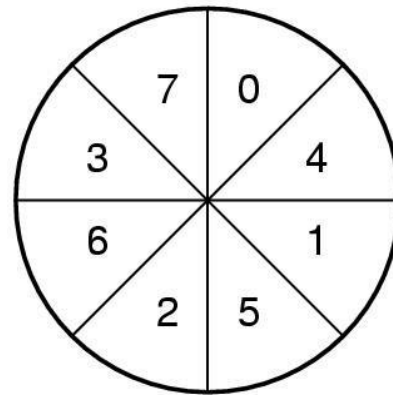
- **Interleaving sectors**

- As a sector is copied into controller buffer, it needs to be copied into main memory.
So next sector should not be adjacent if we wish to avoid waiting for one full rotation



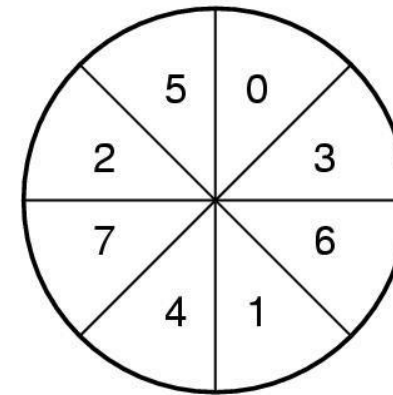
(a)

No interleaving



(b)

Single interleaving



(c)

Double interleaving

- **Disk Scheduling – I/O scheduling**

- Time required to read or write a disk block determined by **3 factors**
 - **Seek time** (moving head to correct cylinder)
 - **Rotational delay** (waiting for correct sector)
 - **Actual transfer time** (data transfer time)
- Seek time dominates
- Error checking is done by controllers
- **Disk Scheduling** - schedule I/O requests arriving for the disk

- **Disk Scheduling Algorithms/Techniques:**

- **FCFS (First Come First Serve)**
- **SSTF (Shortest Seek-Time First)**
- **SCAN (Elevator Algorithm)**
- **C-SCAN (Circular SCAN)**
- **LOOK**
- **C-LOOK**

- **Key Terms Associated with Disk Scheduling**

- **Seek Time:** Seek time is the time taken to locate the disk arm to a specified track where the data is to be read or written. So the disk scheduling algorithm that gives a minimum average seek time is better.
- **Rotational Latency:** Rotational Latency is the time taken by the desired sector of the disk to rotate into a position so that it can access the read/write heads. So the disk scheduling algorithm that gives minimum rotational latency is better.
- **Transfer Time:** Transfer time is the time to transfer the data. It depends on the rotating speed of the disk and the number of bytes to be transferred.
- **Disk Access Time:**

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

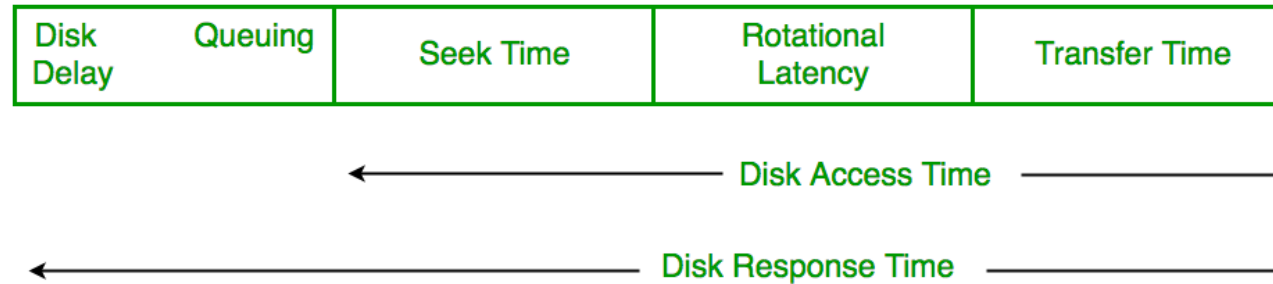
- **Total Seek Time:**

$$\text{Total Seek Time} = \text{Total head Movement} * \text{Seek Time}$$

- **Key Terms Associated with Disk Scheduling**

- **Disk Access Time:**

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$



- **Disk Response Time:** Response Time is the average time spent by a request waiting to perform its I/O operation.
 - **Average Response time** is the response time of all requests.
 - **Variance Response Time** is the measure of how individual requests are serviced with respect to average response time. So the disk scheduling algorithm that gives minimum variance response time is better.

- **Disk Scheduling Algorithms/Techniques - FCFS**

- Basic strategy for moving disk arm across cylinders: First-come-first-served queue

Example:

Disk queue: 98, 183, 37, 122, 14, 124, 65, 67 .

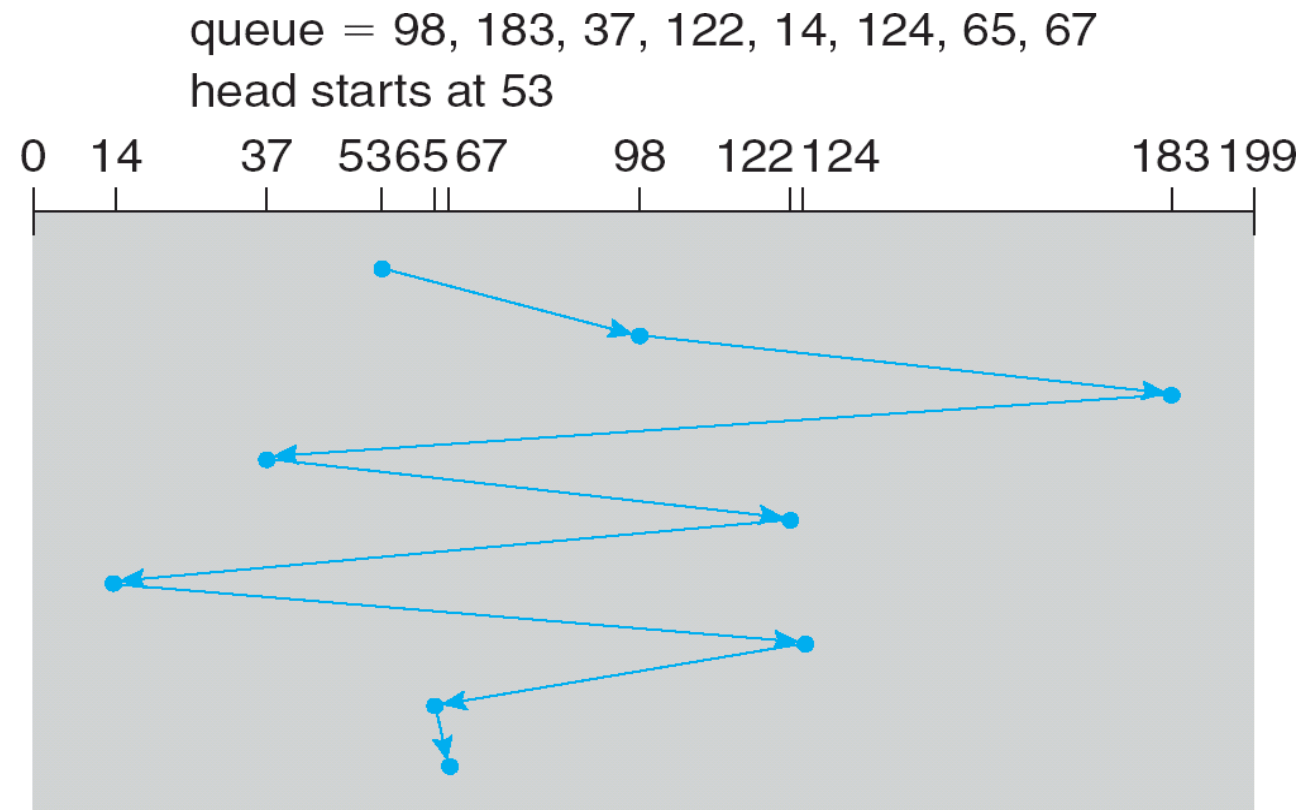
Initial r/w head position: cylinder 53.

Total head movement

$$= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2$$

= 640 tracks

OK for small load, low variance of waiting time



Disk Scheduling Algorithms/Techniques - FCFS

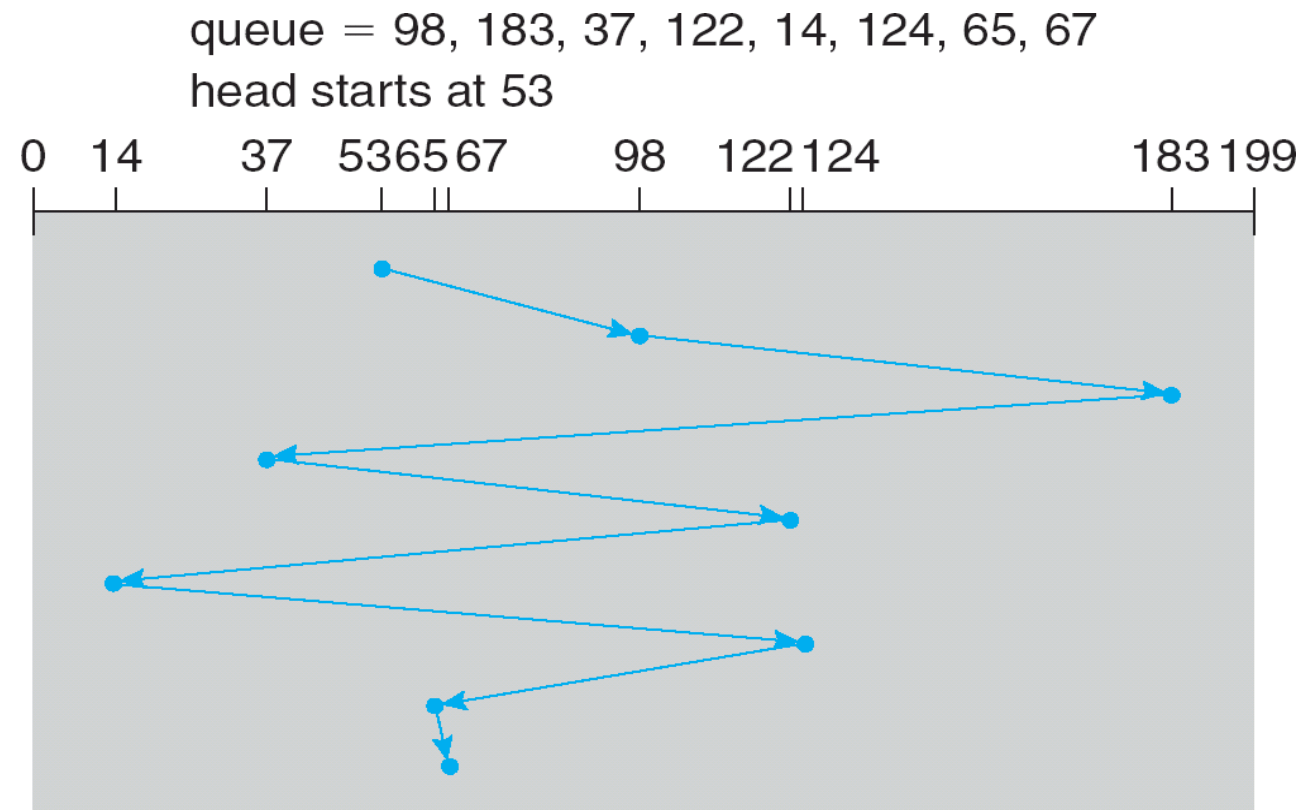
- Basic strategy for moving disk arm across cylinders: First-come-first-served queue

Advantages of FCFS Algorithm

- *Every request gets a fair chance*
- *No indefinite postponement*

Disadvantages of FCFS Algorithm

- *Does not try to optimize seek time*
- *May not provide the best possible service*



Disk Scheduling Algorithms/Techniques – SSTF

- Shortest Seek-Time First (SSF): Pick the closest one next

Example:

Disk queue: 98, 183, 37, 122, 14, 124, 65, 67

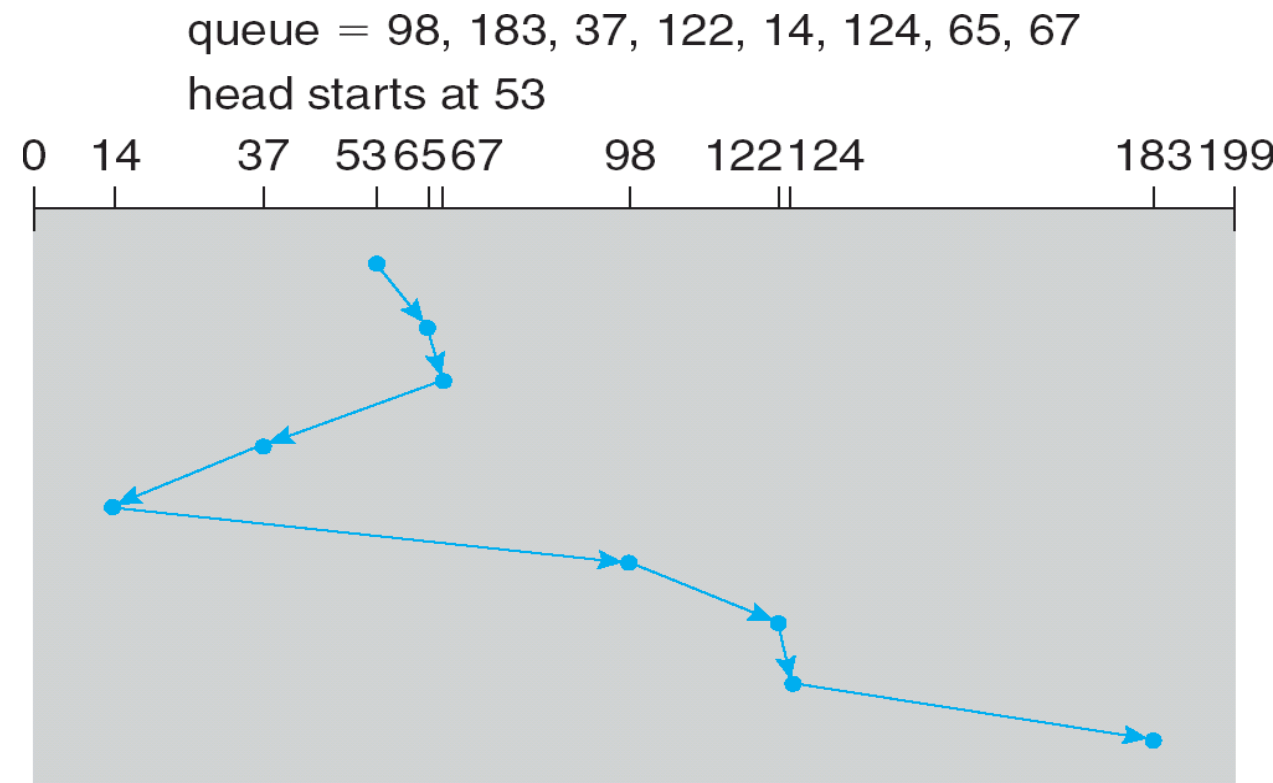
Initial r/w head position: cylinder 53.

Solution:

SSTF ordering will be 65, 67, 37, 14, 98, 122,
124, 183

Total head movement (starting at 53)
= $12 + 2 + 30 + 23 + 84 + 24 + 2 + 59$
= 236 tracks

High variance (starvation possible)



• Disk Scheduling Algorithms/Techniques – SSTF

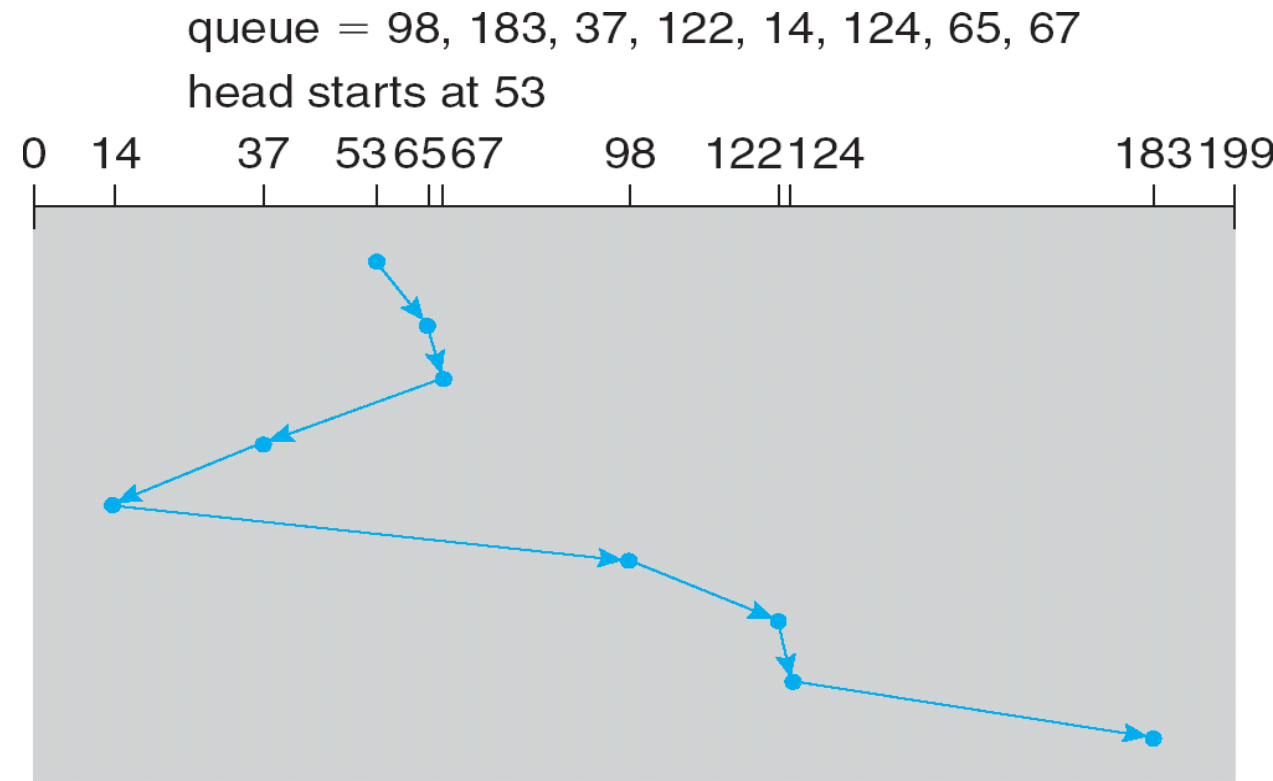
- Shortest Seek-Time First (SSF): Pick the closest one next

Advantages of SSTF Algorithm

- *The average Response Time decreases*
- *Throughput increases*

Disadvantages of SSTF Algorithm

- *Overhead to calculate seek time in advance*
- *Can cause Starvation for a request if it has a higher seek time as compared to incoming requests*
- *The high variance of response time as SSTF favors only some requests*



• Disk Scheduling Algorithms/Techniques – SCAN

- Maintain a direction bit - Pick the request with shortest seek time in current direction first, and change direction only at edges.

Example:

Disk queue: 98, 183, 37, 122, 14, 124, 65, 67

Initial r/w head position: cylinder 53.

Assume head was moving toward 0.

Solution: SCAN ordering will be 37, 14, 0, 65, 67,

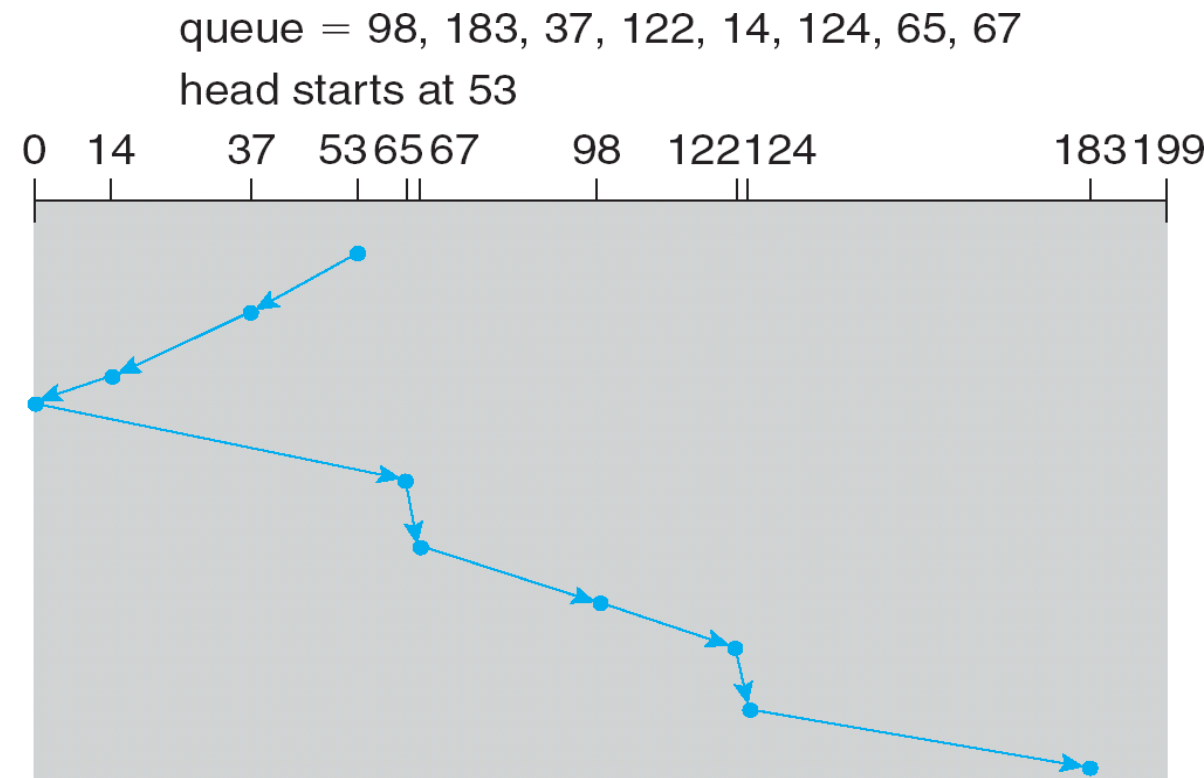
98, 122, 124, 183

Total head movement (starting at 53)

$$= 16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59$$

$$= 236 \text{ tracks}$$

Almost as good as SSF in mean wait time, lower variance.



• Disk Scheduling Algorithms/Techniques – SCAN

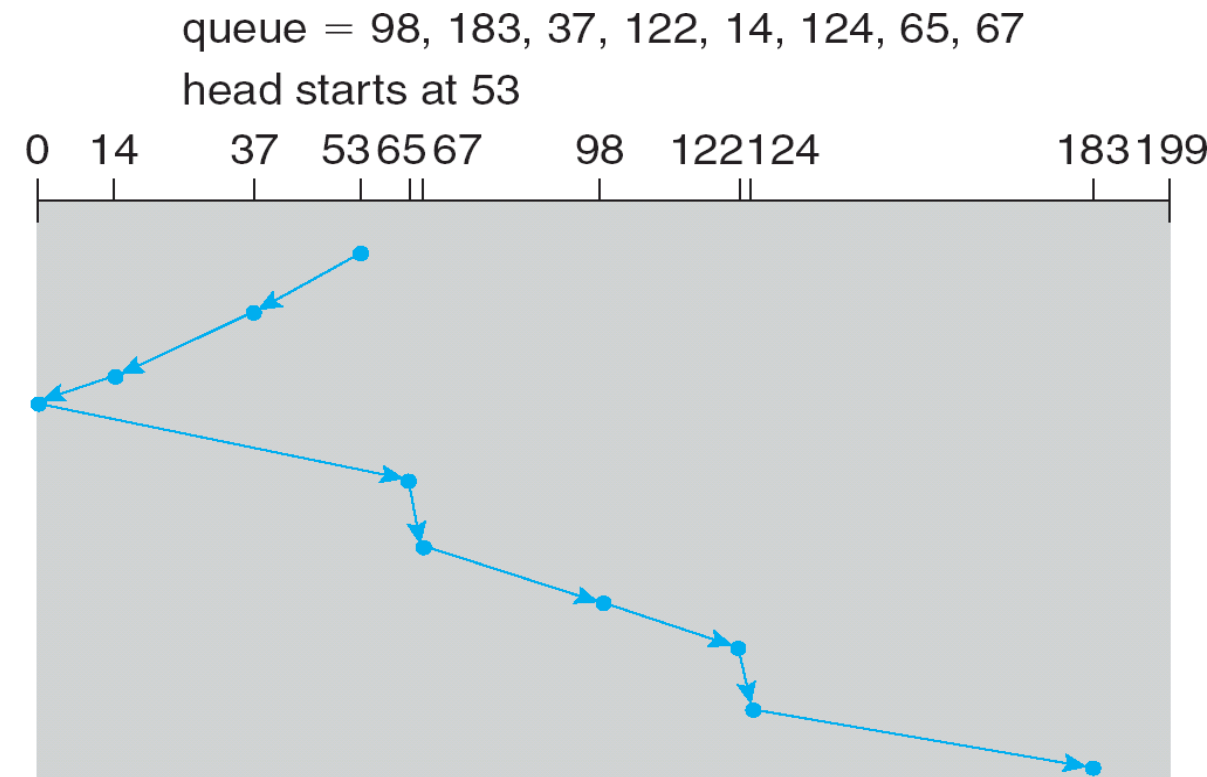
- Maintain a direction bit - Pick the request with shortest seek time in current direction first, and change direction only at edges.

Advantages of SCAN Algorithm

- *High throughput*
- *Low variance of response time*
- *Average response time*

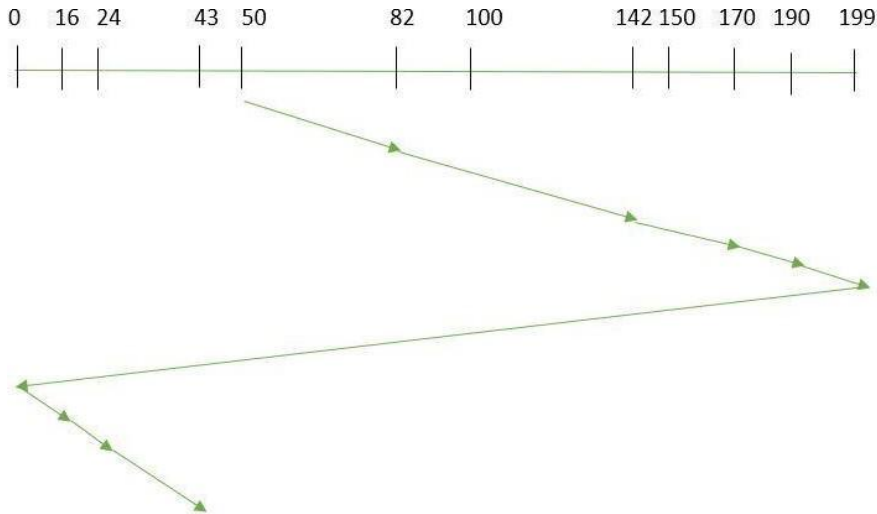
Disadvantages of SCAN Algorithm

- *Long waiting time for requests for locations just visited by disk arm*



• Disk Scheduling Algorithms/Techniques – C – SCAN

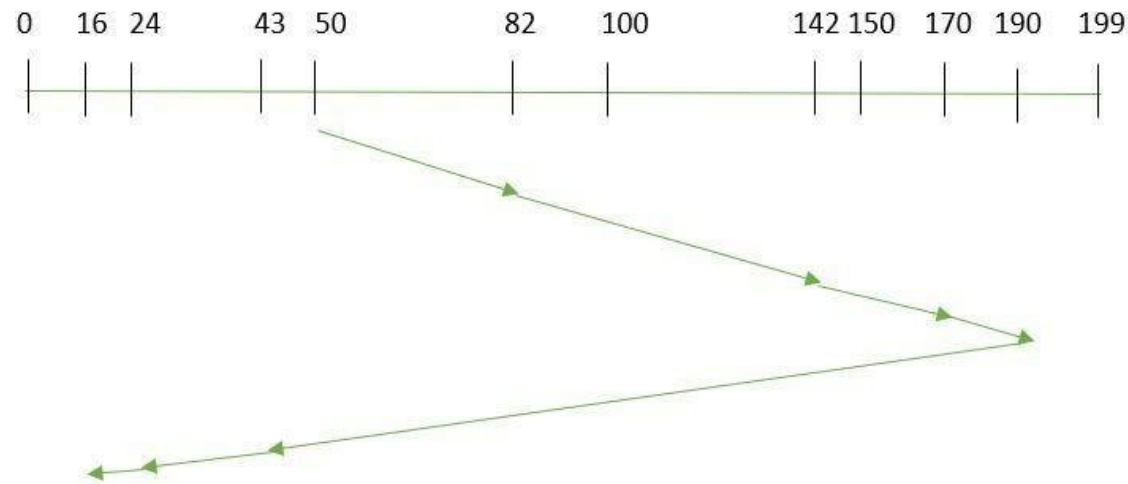
- disk arm **instead of reversing its direction** goes to the other end of the disk and starts servicing the requests from there.
- So, the disk arm moves in a **circular fashion** and this algorithm is also similar to the SCAN algorithm hence it is known as **C-SCAN (Circular SCAN)**.



Provides more uniform wait time compared to SCAN.

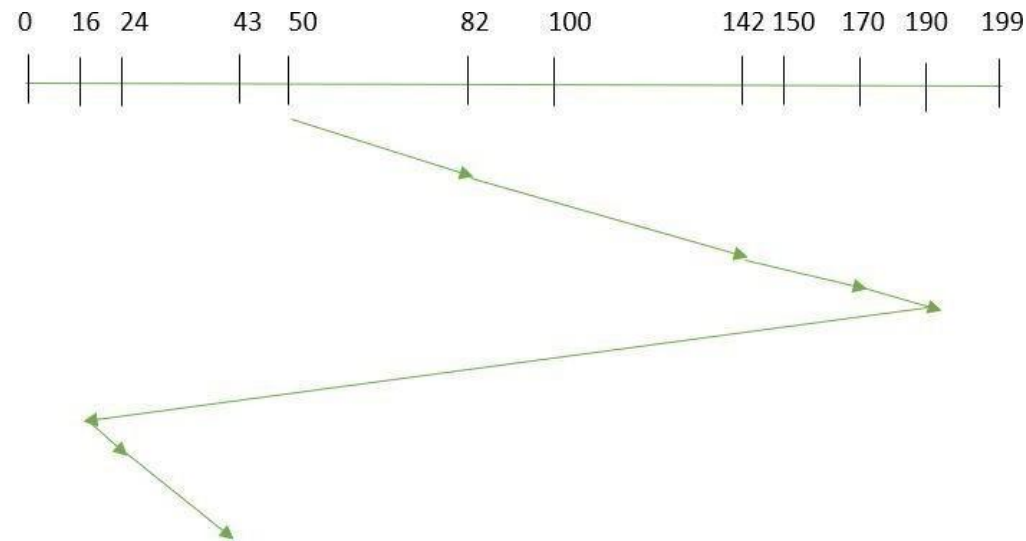
• Disk Scheduling Algorithms/Techniques – LOOK

- similar to the SCAN - disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only.
- Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.



• Disk Scheduling Algorithms/Techniques – C-LOOK

- similar to the C-SCAN - disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request.
- Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.



Reliability of Memory Systems

- **Fault tolerance for Disks**

- Disks can have lots of errors
 - Corrupted bits
 - Corrupted sectors
 - Manufacturing defects
 - Disk crashes
- Many levels of protection implemented
 - Error correction within sectors
 - Extra sectors within a disk
 - Mirroring
 - RAID (redundancy and parallelism)

External Storage Systems - RAID

- Disk access times are still slow compared to CPU/memory
- **Natural solution:** Read concurrently from multiple disks
- **RAID (Redundant Array of Independent Disks)** is a technique that makes use of a **combination of multiple disks** instead of using a single disk for increased
 - performance
 - data redundancy or
 - Both
- **RAID Controller** talks to OS just **like a device driver** and manages a suite of disks to improve reliability and performance
- A RAID system consists of **two or more drives working in parallel**.

- RAID (redundant array of independent disks) is a way of storing the same data in different places on multiple hard disks or solid-state drives (SSDs) to protect data in the case of a drive failure.
- There are different RAID levels, however, and **not all have the goal of providing redundancy.**
- **RAID arrays** appear to the operating system (OS) as a **single logical drive.**
- RAID employs the **techniques of disk mirroring or disk striping.**
- **Mirroring** will copy identical data onto more than one drive.
- **Striping partitions** help spread data over multiple disk drives.

- **Striping**

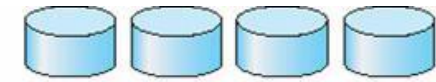
- Organize **logically consecutive chunks of data** across **different disks** to improve performance
- Striping can be at **bit-level, byte-level, block-level**
- **Incoming request** is split into all relevant disks by **RAID controller**, and their outputs are put together

- **Redundancy through duplication**

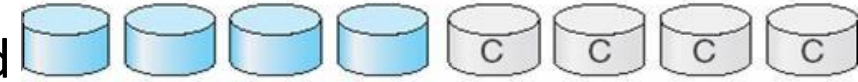
- Multiple requests can be **load-balanced**
- **Error recovery** on faults

RAID Levels

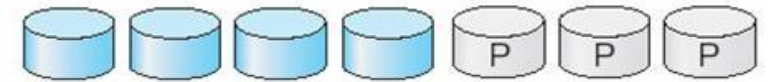
- Many different ways of distributing data have been standardized into various RAID levels.
- Each RAID level is offering a trade-off of data protection, system performance, and storage space.
- The number of levels has since expanded and has been broken into **three categories: standard, nested and nonstandard RAID levels.**
- The most popular and standard RAID levels
 - RAID 0
 - RAID 1
 - RAID 2
 - RAID 3
 - RAID 4
 - RAID 5
 - RAID 6



(a) RAID 0: non-redundant striping.



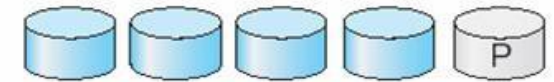
(b) RAID 1: mirrored disks.



(c) RAID 2: memory-style error-correcting codes.



(d) RAID 3: bit-interleaved parity.



(e) RAID 4: block-interleaved parity.



(f) RAID 5: block-interleaved distributed parity.



(g) RAID 6: P + Q redundancy.

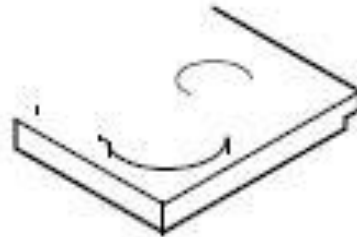
RAID Level 0(Non-Redundant Striping)

- It allows to write data across multiple hard drives instead of one hard drive RAID 0 partitions each hard drive storage space into 64 KB stripes.
- These stripes are interleaved in a repeated sequential manner. The part of the stripe on a single hard drive is called a stripe element.
- For example, in a four-disk system using only RAID 0, segment 1 is written to disk 1, segment 2 is written to disk 2, and so on. RAID 0 enhances performance because multiple hard drives are accessed simultaneously, but it does not provide data redundancy

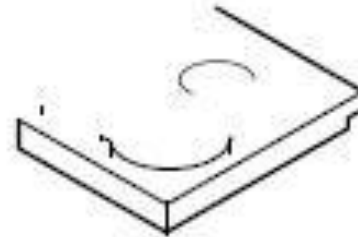
•**Fault Tolerance** - None

•**Advantage** - Improved performance, extra storage

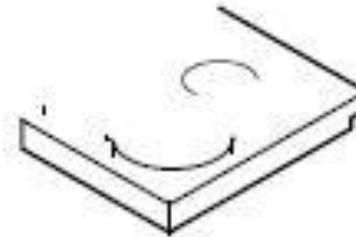
•**Disadvantage** - Should not be used for critical data Data Loss occurs with any drive failure.



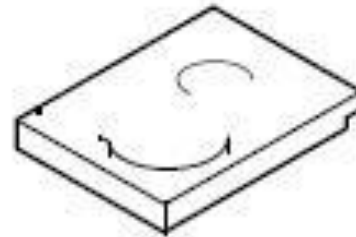
stripe element 1
stripe element 5
stripe element 9



stripe element 2
stripe element 6
stripe element 10



stripe element 3
stripe element 7
stripe element 11



stripe element 4
stripe element 8
stripe element 12

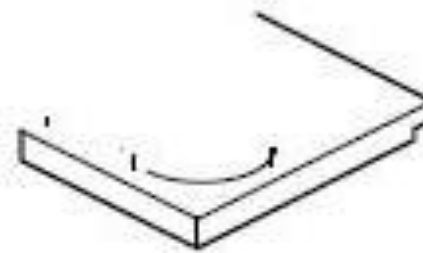
RAID Level 1 (Mirrored disks)

- With RAID 1, data written to one disk is simultaneously written to another disk.
- If one disk fails, the contents of the other disk can be used to run the system and rebuild the failed hard drive.
- The primary advantage of RAID 1 is that it provides 100 percent data redundancy.
- Because the contents of the disk are written to a second disk, the system can sustain the failure of one disk. Both disks contain the same data always. Either hard drive can act as the operational hard drive

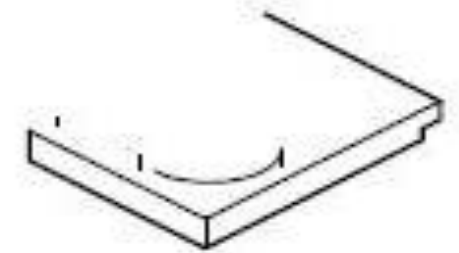
•**Fault Tolerance** - Disk errors, Single disk failure

•**Advantage** - High read performance, Fast recovery after drive failure, Data redundancy

•**Disadvantage** - High disk overhead, Limited capacity



stripe element 1
stripe element 2
stripe element 3
stripe element 4

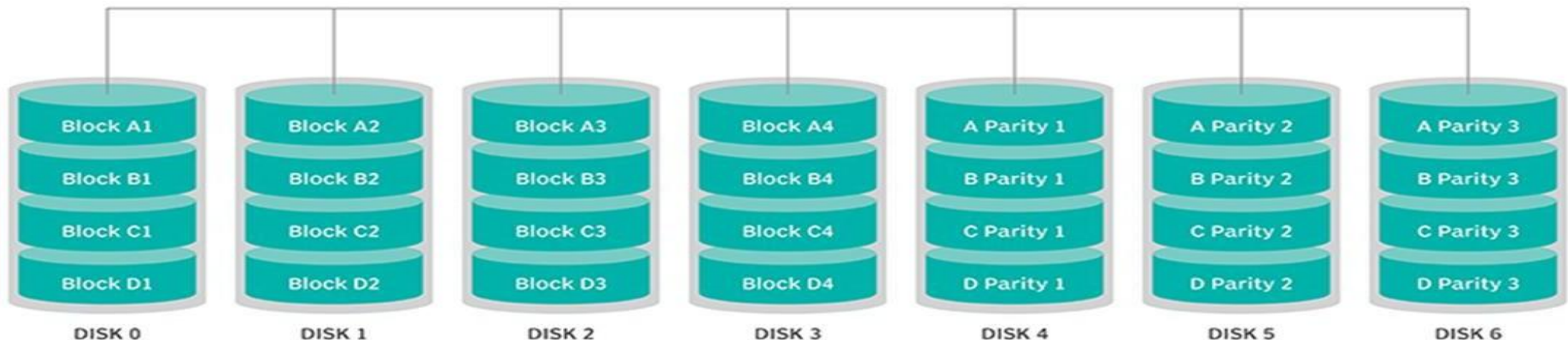


stripe element 1 duplicated
stripe element 2 duplicated
stripe element 3 duplicated
stripe element 4 duplicated

RAID Level 2 –bit-level striping(Memory-style error correcting code organization)

- This configuration uses **striping across disks**, with **some disks storing error checking and correcting (ECC) information**.
- RAID 2 also **uses a dedicated Hamming Code parity**, a linear form of ECC
- Called Uses **bit-level striping**
- Some of the bits used for parity or other error correction
- **Sample:** *4-bits of data encoded by additional 3-bits of ECC, and resulting 7 bits stored on 7 different disks*

RAID 2



- **RAID Level 2 - Advantages**

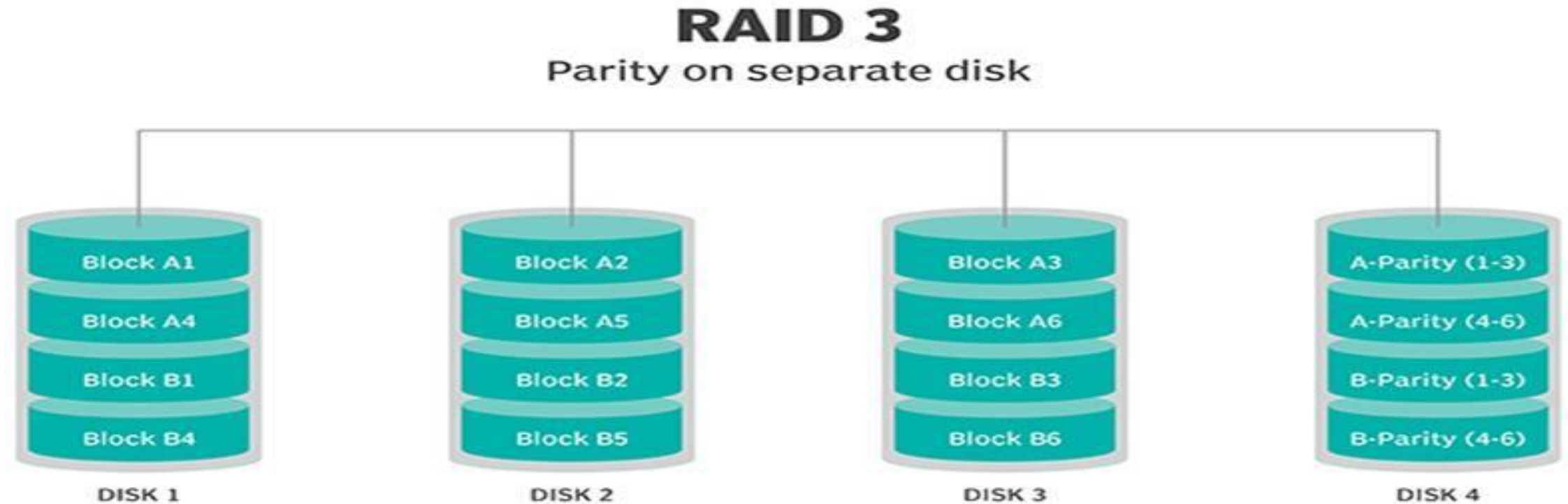
- Great parallelism for read as well as write
- Even if one disk crashes, all data can be recovered (due to ECC)

- **Disadvantages**

- Drives have to be synchronized
- Overhead in putting together reads from different disks
- Crash recovery much slower (compared to RAID-1)

RAID Level 3(byte striping with parity)

- With RAID 3, data is divided into bit or byte units and written across multiple dedicated data disk drives.
- Parity information is created for each of separate data section and written to a dedicated parity drive.
- All disk drives can be accessed in parallel all the time and the data can be transferred in bulk, ensuring high-speed data transfer.



RAID Level 3 – How Parity Works?

- Truth table for XOR (same as parity)

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

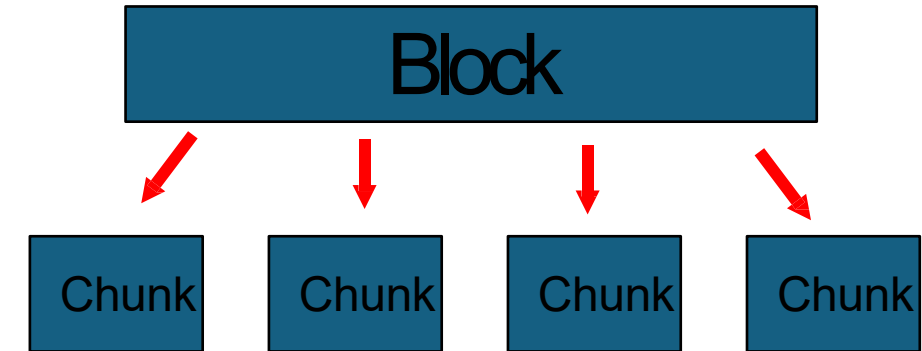
- Recovering from a disk failure
 - Small RAID level 3 array with data disks D0 and D1 and parity disk P can tolerate failure of either D0 or D1

D0	D1	P
0	0	0
0	1	1
1	0	1
1	1	0

$D1 \oplus P = D0$	$D0 \oplus P = D1$
0	0
0	1
1	0
1	1

RAID Level 3 – How it Works?

- Assume we have $N + 1$ disks
- Each block is partitioned into N equal chunks

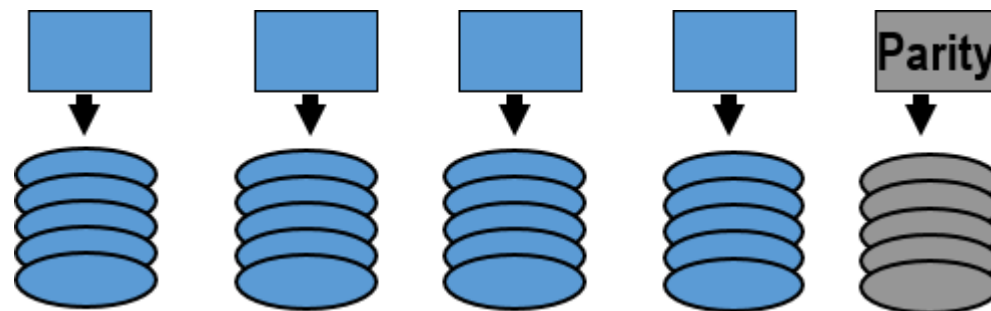


- XOR data chunks to compute the parity chunk

$N = 4$ in example



- Each chunk is written into a separate disk



RAID Level 4(block striping with parity)

- RAID 4 features data re-creation through a combination of RAID 0 striping and the use of a dedicated parity disk. Data is divided into unit blocks and kept on dedicated data disk drives while parity data is kept on a single dedicated parity disk.
- When updating it is necessary to pre-read both the existing and parity data and write updated parity data when the update is complete. This process is referred to as having a "write penalty".
- It is usually impossible to implement this process in business environments as the dedicated parity disk becomes a bottleneck during traffic surges and performance suffers.

RAID 4



RAID Level 5

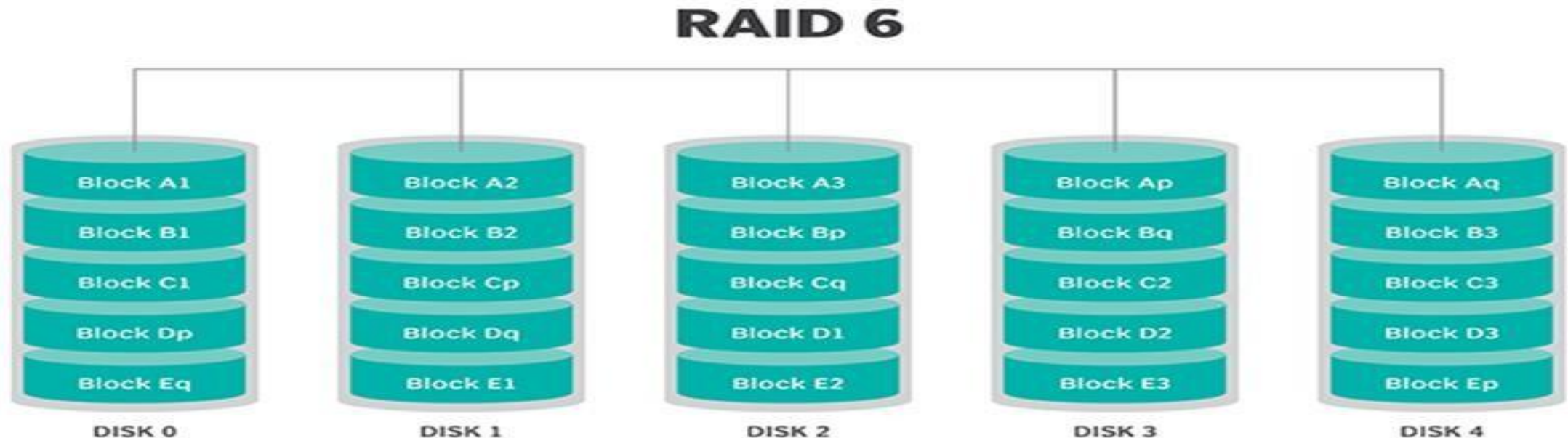
- RAID 5 is the most popular RAID technology in use today. It uses a technique that avoids the concentration of I/O on a dedicated parity disk, that occurs with RAID 4. Although RAID 5 divides the data and creates parity information similar to RAID 4, unlike RAID 4 the parity data is written separately across multiple disks.
- "Write penalty" still occurs as existing data must be pre-read before update and parity data has to be updated after the data is written.
- However, RAID 5 enables multiple write orders to be implemented concurrently because updated parity data is dispersed across the multiple disks. This feature ensures higher performance compared with RAID 4.

RAID 5



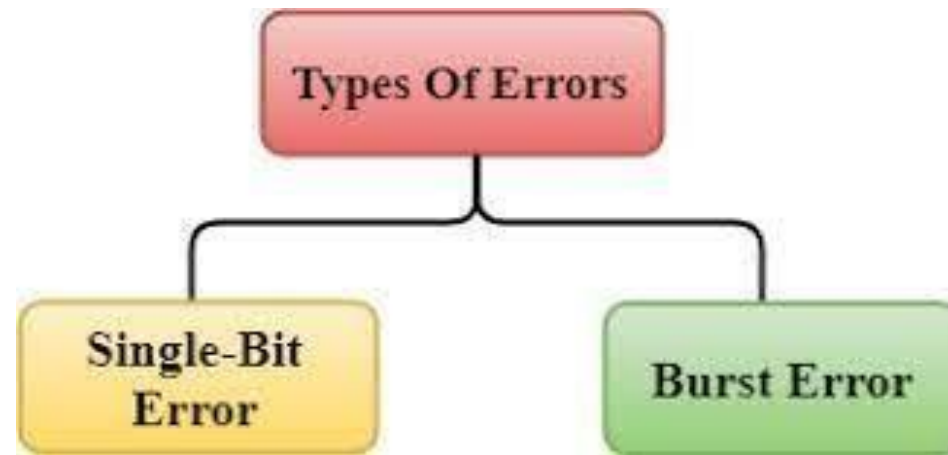
RAID Level 6

- RAID 6 deploys two parity records to different disk drives (double parity) enabling two simultaneous disk drive failures in the same RAID group to be recovered.
- RAID 6 where parity updates are allocated separately across multiple disks, as well as RAID 5, are able to implement multiple write orders at the same time. This feature ensures higher performance when compared to RAID 4 technology.



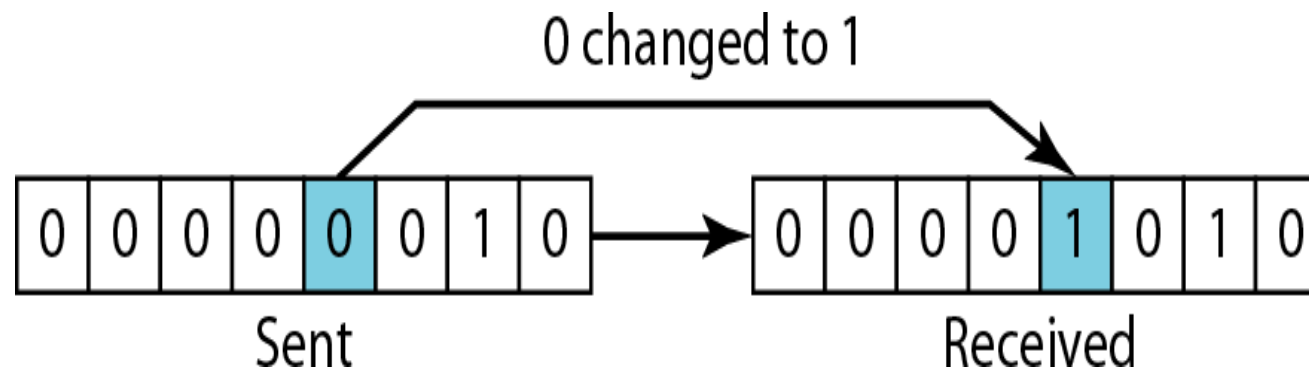
Error detecting and error correcting systems

- Error detection and correction code plays an important role in the transmission of data from one source to another.
- The noise also gets added into the data when it transmits from one system to another, which causes errors in the received binary data at other systems.
- The bits of the data may change(either 0 to 1 or 1 to 0) during transmission.
- It is impossible to avoid the interference of noise, but it is possible to get back the original data.
- For this purpose, we first need to detect either an error is present or not using error detection codes.
- If the error is present in the code, then we will correct it with the help of error correction codes.



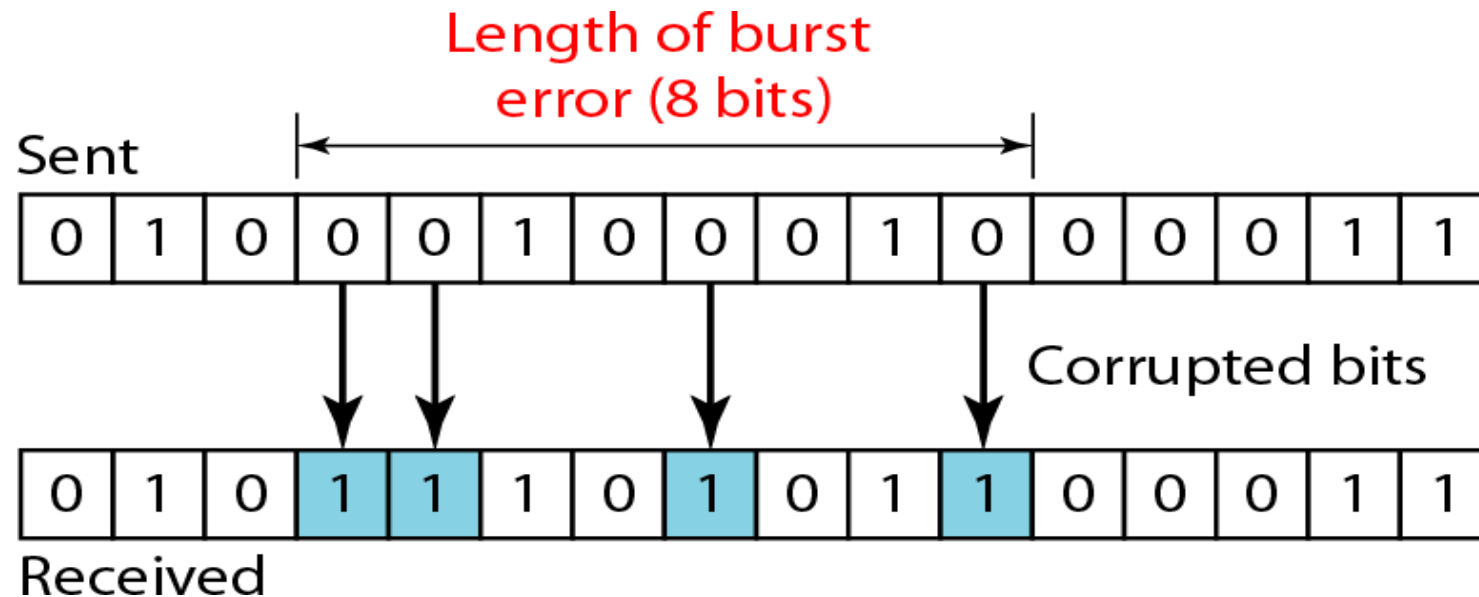
Single-bit error

In a single-bit error, only 1 bit in the data unit has changed.



Burst error of length 8

A burst error means that 2 or more bits in the data unit have changed.



Error Detection and Correction Techniques

```
graph TD; A[Error Detection and Correction Techniques] --> B[1 Parity Code]; A --> C[2 Checksum]; A --> D[3 Cyclic Redundancy Check(CRC)]; A --> E[4 Hamming Code];
```

1

Parity
Code

2

Checksum

3

Cyclic
Redundancy
Check(CRC)

4

Hamming
Code

To detect or correct errors, we need to send extra (redundant) bits with data.

Error detection codes – are used to detect the errors present in the received data bitstream. These codes contain some bits, which are appended to the original bit stream. These codes detect the error, if it is occurred during transmission of the original data

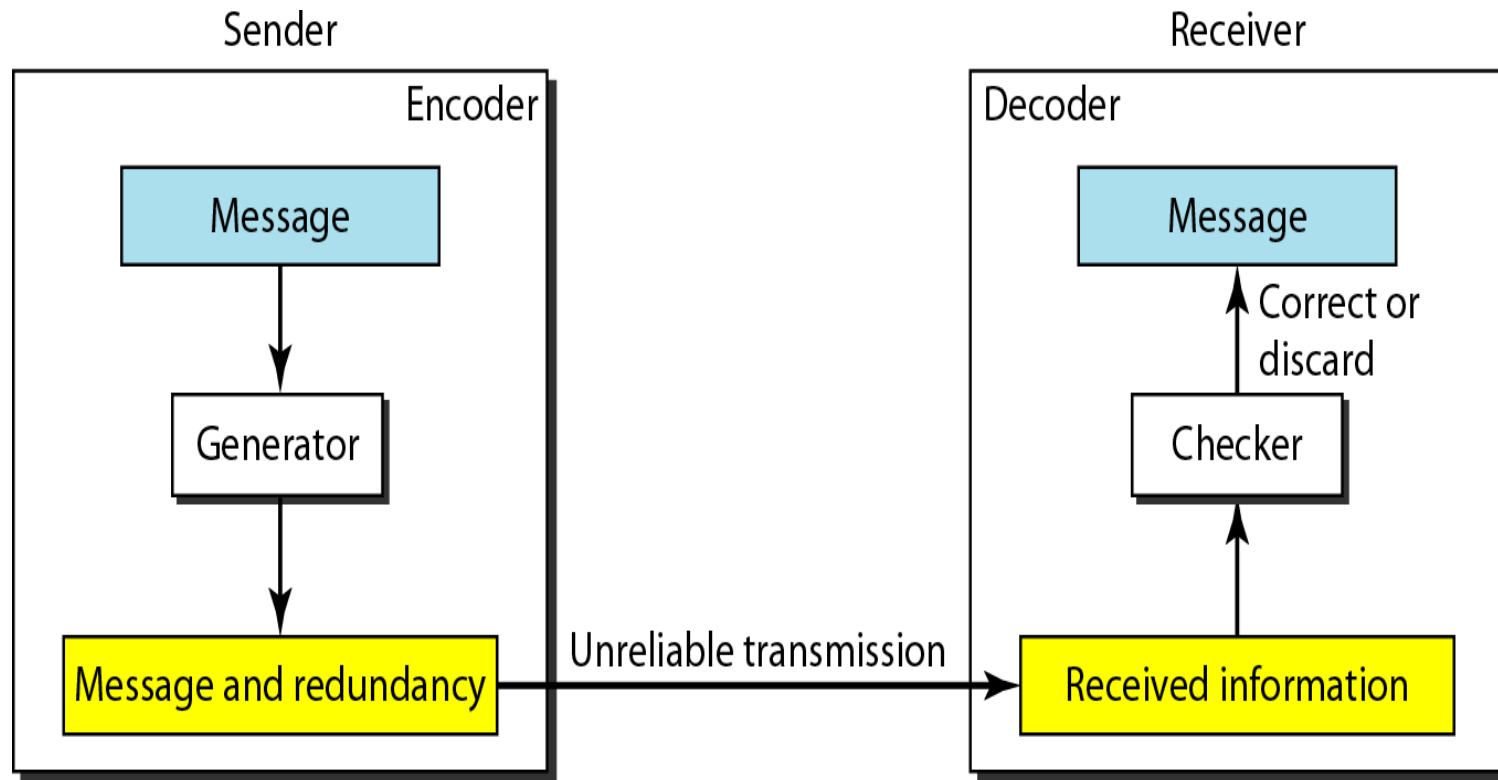
Example – Parity code, Hamming code.

Error correction codes – are used to correct the errors present in the received data bitstream so that, we will get the original data. Error correction codes also use the similar strategy of error detection codes.

Example – Hamming code.

To detect or correct errors, we need to send extra (redundant) bits with data.

The structure of encoder and decoder



Parity Code

- It is easy to **append one parity bit either to the left of MSB or to the right of LSB** of original bit stream. There are two types of parity codes, namely even parity code and odd parity code based on the type of parity being chosen.

Even Parity Code

- The **value of even parity bit should be zero**, if **even number of ones present** in the binary code. Otherwise, **it should be one**.

Binary Code	Even Parity bit	Even Parity Code
000	0	0000
001	1	0011
010	1	0101
011	0	0110
100	1	1001
101	0	1010
110	0	1100
111	1	1111

Odd Parity Code

- The value of odd parity bit should be zero, if odd number of ones present in the binary code. Otherwise, it should be one..

Binary Code	Odd Parity bit	Odd Parity Code
000	1	0001
001	0	0010
010	0	0100
011	1	0111
100	0	1000
101	1	1011
110	1	1101
111	0	1110

Hamming Code

- Most common types of error-correcting codes used in RAM
- devised by R. W. Hamming.
- In the Hamming code, **k parity bits** are added to an **n-bit data word**, forming a new word of **n + k bits**.
- The bit **positions** are numbered in sequence from **1 to n + k**.
- **Positions** numbered with **powers of two** are reserved for the parity bits. ($2^0, 2^1, 2^2, 2^3, 2^4 \dots$)
- **Remaining** bits are the **data** bits.

Hamming Code

1. Number of parity bit calculation

$$2^r \geq m + r + 1$$

where, r = redundant bit (or) parity bit,

m = data bit

Example: Consider hamming code of data is 1110 and number of data bits (m) = 4, calculate number of parity bits.

If $r=1 \rightarrow 2^1 \geq 4 + 1 + 1 = 2 \geq 6 \rightarrow$ equation is not satisfied-**wrong** assumption

If $r=2 \rightarrow 2^2 \geq 4 + 2 + 1 = 4 \geq 7 \rightarrow$ equation is not satisfied-**wrong** assumption

If $r=3 \rightarrow 2^3 \geq 4 + 3 + 1 = 8 \geq 8 \rightarrow$ equation is satisfied-**correct** assumption

Result is if m (data bits) is 4 and r (parity bits) is 3. Total bits=7

Hamming Code

2. Placement of Parity Bits

Power of Two	2^0	2^1		2^2			
Bit Location	1	2	3	4	5	6	7
Bits	P1	P2	M1	P3	M2	M3	M4

Placement of Parity Bits

P1, P2, and P3 are parity bits.

M1, M2, M3, and M4 are message bits

Hamming Code

3. Hamming Code with Even Parity

- Let us determine Hamming code for the message to be transmitted **1110**.
- Once we get the message to be transmitted, we can fix the places for **M1**, **M2**, **M3**, and **M4**, as shown below.

Power of Two	2^0	2^1		2^2			
Bit Location	1	2	3	4	5	6	7
Bits	P1	P2	1	P3	1	1	0

Even Bit Parity

Hamming Code

4. Rules to Determine the Parity Sequence

	P3	P2	P1	Decimal Equivalent
	0	0	0	0
P1(1,3,5,7) ←	0	0	1	1
P2(2,3,6,7)	0	1	0	2
P3(4,5,6,7)	0	1	1	3
	1	0	0	4
	1	0	1	5
	1	1	0	6
	1	1	1	7

$P1=(1,3,5,7) \rightarrow (P1,1,1,0) \rightarrow \text{Even number of 1's} \rightarrow P1=0$

$P2=(2,3,6,7) \rightarrow (P2,1,1,0) \rightarrow \text{Even number of 1's} \rightarrow P2=0$

$P3=(4,5,6,7) \rightarrow (P3,1,1,0) \rightarrow \text{Even number of 1's} \rightarrow P3=0$

Hamming Code

Problem 1: (Case 1)

Let us assume that the code transmitted in (7,4) format is 1110000. Check the error with even parity bit.

(i) Parity bits = 3

Power of Two	2^0	2^1		2^2			
Bit Location	1	2	3	4	5	6	7
Bits	P1	P2	M1	P3	M2	M3	M4

Placement of Parity Bits

Consider the following table.

(ii)

Location	1	2	3	4	5	6	7
Bits	P1	P2	M1	P3	M2	M3	M4
P1(1,3,5,7)	1	1	1	0	0	0	0
P2(2,3,6,7)							
P3(4,5,6,7)							

Hamming Code Error Detection

(iii)

Error					Even Parity
E1(1,3,5,7)	1	1	0	0	E1=0
E2(2,3,6,7)	1	1	0	0	E2=0
E3(4,5,6,7)	0	0	0	0	E3=0
Error Code	(E3E2E1)=(000)				

Hamming Code Error Detection Bits

It is clear that the error code is 000, which means that **no error has occurred in the transmitted data**

Hamming Code

Case 2:

Let us assume that the code transmitted in (7,4) format is 1111000 .Check the error with even parity bit.

(i) Parity bits =3

Power of Two	2^0	2^1		2^2			
Bit Location	1	2	3	4	5	6	7
Bits	P1	P2	M1	P3	M2	M3	M4

Placement of Parity Bits

Consider the following table.

(ii)

Location	1	2	3	4	5	6	7
Bits	P1	P2	M1	P3	M2	M3	M4
P1(1,3,5,7)	1	1	1	1	0	0	0
P2(2,3,6,7)							
P3(4,5,6,7)							

Hamming Code Error Detection

(iii)

Error					Even Parity
E1(1,3,5,7)	1	1	0	0	E1= 0
E2(2,3,6,7)	1	1	0	0	E2= 0
E3(4,5,6,7)	1	0	0	0	E3= 1
Error Code	(E3E2E1)=(100)				

Hamming Code Error Detection Bits

The decimal value of the error code is $(100)_2 \rightarrow$ decimal: 4 so **Error is in bit 4**
 Error Correction: Since the value of bit 4 is 1, we flip it to 0.
 The corrected data is **1110000**.